

8byte.ai DevOps Assignment - Complete Project Explanation

Project Overview

This project demonstrates a complete DevOps deployment pipeline on AWS using Infrastructure as Code, Containerization, CI/CD, Security Best Practices, and Monitoring.

The application is a Python Flask web application containerized using Docker and deployed on AWS ECS Fargate. The entire infrastructure is provisioned using Terraform, and deployment is automated through GitHub Actions.

Business Requirement

The objective of the project is to:

- Deploy a Flask application on AWS
 - Use containerization
 - Automate infrastructure provisioning
 - Implement CI/CD pipeline
 - Follow security best practices
 - Enable monitoring and logging
-

High Level Architecture

User Request ↓ Application Load Balancer (ALB) ↓ ECS Fargate Service ↓ Flask Application Container ↓ PostgreSQL Database (RDS)

Supporting Services:

- ECR (Container Registry)
 - CloudWatch (Logs & Monitoring)
 - Parameter Store (Secrets Management)
 - GitHub Actions (CI/CD)
 - Terraform (Infrastructure Provisioning)
-

Technologies Used

Infrastructure:

- AWS
- Terraform

Containerization:

- Docker
- Amazon ECR

Container Orchestration:

- Amazon ECS Fargate

Database:

- PostgreSQL RDS

CI/CD:

- GitHub Actions

Monitoring:

- CloudWatch

Security:

- IAM Roles
- Security Groups
- AWS Parameter Store

Why Terraform?

Terraform is used because:

- Infrastructure becomes reusable
- Infrastructure becomes version controlled
- Infrastructure can be recreated anytime
- Reduces manual configuration

Instead of creating resources manually from AWS Console, everything is created through code.

Infrastructure Created Using Terraform

Terraform provisions:

1. VPC
2. Public Subnets
3. Private Subnets
4. Internet Gateway
5. Route Tables
6. Security Groups
7. ECS Cluster
8. ECS Service
9. ECS Task Definition
10. Application Load Balancer
11. Target Group
12. ECR Repository
13. PostgreSQL RDS
14. CloudWatch Log Group
15. IAM Roles
16. Parameter Store Secrets

VPC Design

Public Subnets:

- Application Load Balancer

Private Subnets:

- ECS Fargate Tasks
- PostgreSQL Database

Reason:

Application should be accessible from the internet.

Database should remain private and secure.

Docker Implementation

The Flask application is containerized using Docker.

Docker Benefits:

- Consistent environment
- Easy deployment
- Scalability
- Portability

Dockerfile performs:

1. Uses Python base image
2. Installs dependencies
3. Copies application code
4. Exposes application port
5. Starts application using Gunicorn

Why Gunicorn?

Gunicorn is a production-grade WSGI server.

Benefits:

- Better performance
- Handles multiple requests
- Production ready

Amazon ECR

ECR is used to store Docker images.

Process:

Developer ↓ Docker Build ↓ Push Image to ECR ↓ ECS Pulls Image from ECR

Benefits:

- Secure image storage
 - AWS native integration
 - Version management
-

ECS Fargate Deployment

Application is deployed on ECS Fargate.

Why Fargate?

- No EC2 management
- No patching
- No server maintenance
- Serverless container execution

AWS manages the infrastructure automatically.

ECS Components

ECS Cluster

Logical grouping of services and tasks.

Task Definition

Blueprint of container deployment.

Contains:

- Container Image
- CPU
- Memory
- Environment Variables
- Logging Configuration
- Port Mapping

ECS Service

Maintains desired number of running tasks.

If a container crashes:

ECS automatically launches a replacement container.

Application Load Balancer

ALB is placed in front of ECS.

Responsibilities:

- Distributes traffic
- Health Checks
- High Availability

User traffic always reaches ALB first.

PostgreSQL Database

Database is hosted on Amazon RDS.

Why RDS?

- Managed database service
- Automated backups
- Monitoring support
- High availability options

Database is deployed in private subnet.

Reason:

Database should never be exposed publicly.

Secret Management

Database password is stored in:

AWS Systems Manager Parameter Store

Type:

SecureString

Benefits:

- Encryption

- Centralized management
- No hardcoded passwords

Application retrieves secret securely during runtime.

CI/CD Pipeline

GitHub Actions is used for automation.

Workflow:

Developer Push Code ↓ GitHub Actions Triggered ↓ Build Docker Image ↓ Run Security Scan (Trivy) ↓ Push Image to ECR ↓ Deploy to ECS ↓ Rolling Update

Why Trivy?

Trivy scans Docker images for vulnerabilities.

Benefits:

- Detects security issues
 - Prevents insecure deployments
 - Improves application security
-

Monitoring

CloudWatch is used for:

- Application Logs
- ECS Metrics
- ALB Metrics
- Error Tracking

Metrics monitored:

- CPU Utilization
 - Memory Utilization
 - Request Count
 - Latency
 - Container Health
-

Health Check Endpoint

Endpoint:

/health

Purpose:

Allows ALB and monitoring systems to verify application health.

Healthy response indicates application is working correctly.

Application Endpoints

/

Returns application status.

/health

Returns health status.

/db

Checks database connectivity.

Security Best Practices Implemented

1. Private RDS Deployment
 2. Security Groups
 3. IAM Roles
 4. Parameter Store Secrets
 5. Container Vulnerability Scanning
 6. Least Privilege Access
-

Challenges Faced

One common challenge was secret retrieval from Parameter Store.

Solution:

Attached necessary IAM permissions to ECS Task Execution Role and enabled KMS decrypt permissions.

Future Improvements

1. Auto Scaling
 2. Blue-Green Deployment
 3. Multi-AZ RDS
 4. AWS WAF
 5. Integration Testing
 6. Disaster Recovery Strategy
 7. Enhanced Monitoring Dashboards
-

Final Project Summary

This project demonstrates a complete production-style DevOps implementation using Terraform, Docker, ECS Fargate, ECR, RDS PostgreSQL, CloudWatch, Parameter Store, and GitHub Actions CI/CD pipeline.

The entire infrastructure is automated, secure, scalable, and follows DevOps best practices.